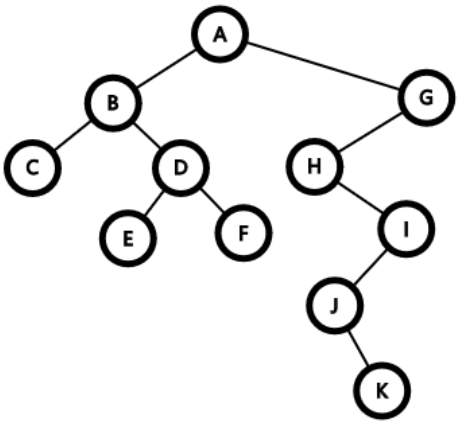
1.

最终序列为:

```
先序: A B C D E F G H I J K
中序: C B E D F A H J K I G
后序: C E F D B K J I H G A
```

树为:



2.

对树结构归纳.

(a) 当根是叶子时, 显然成立;

(b) 当根只有一个孩子, 则对此孩子的子树归纳可知成立;

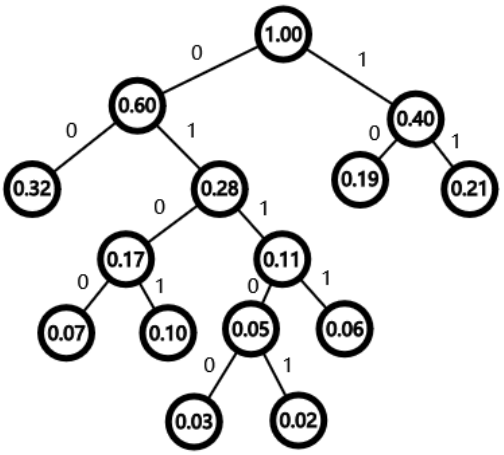
(c) 若根节点有左右孩子 l, r, 各序列总会先枚举完 l 子树中的所有结点, 再进入 r 子树, 所以在三种序列中, l 子树内结点总出现在 r 子树内结点之前. 最后对 l 和 r 子树分别归纳可知成立.

3.

(1) 根据 Huffman 树的贪心性质, 不可能出现只具有一个孩子的结点, 否则可以删除该结点并将其唯一孩子向其父亲连边, 使得编码仍然合法的条件下变短. 所以

$$2(n_{\text{total}} - n_{\text{leaf}}) + 1 = n_{\text{total}} = 199 \Rightarrow n_{\text{leaf}} = 100.$$

(2) 如图构造 Huffman 树:



不妨设字母为 A~H, 则编码为

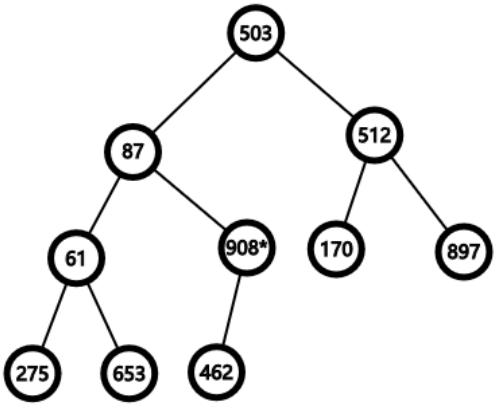
字母	出现概率	编码
A	0.07	`0100`
B	0.19	`10`
C	0.02	`01101`
D	0.06	`0111`
E	0.32	`00`
F	0.03	`01100`
G	0.21	`11`
H	0.10	`0101`

(3) 相较于 Huffman 编码, 使用三位二进制编码的优劣为:

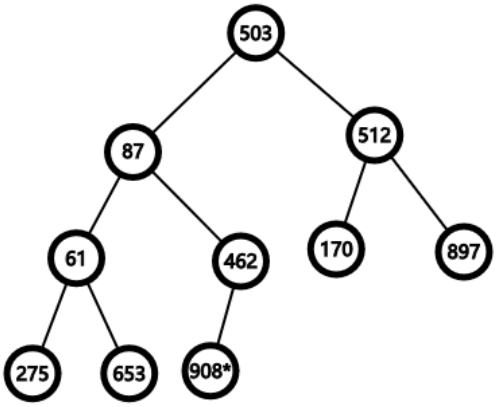
- **优势**
  - 编码映射更简单, 易于实现与验证;
  - 单字编码长度固定, 易于对长电文进行截取, 翻译等处理, 更易纠错;
- **劣势**
  - 编码期望长度更大.

4.

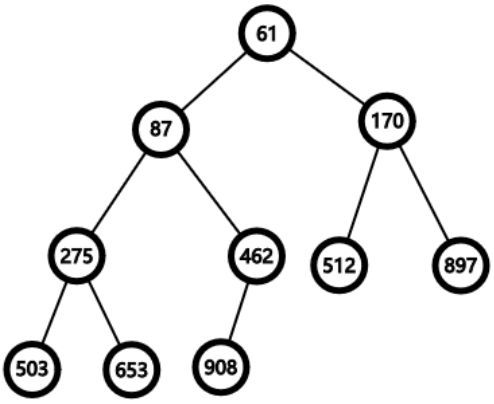
(1) 第一步, 将序列视作二叉堆, 并从 $n/2 = 5$ 号结点开始 sift down 调整.



第一次 sift down 后变为



同理, 最后构建出的二叉堆为



(2) 可能在任何一个叶结点处.

(3) 在建堆时, 我们依次遍历结点 $n/2$ ("/" 除号均指下取整除法) 到 1, 进行 sift down 操作. 设叶结点的高度为 0, 则对高度为 h 的结点 sift down 至多进行 h 次比较. 因此, 总比较次数的最大值就是 n 个结点的满二叉树的各结点高度之和, 记为 $f(n)$. 可以写出

$$f(2^1-1)=0, \quad f(2^{w+1}-1)=2f(2^w-1)+w.$$

所以

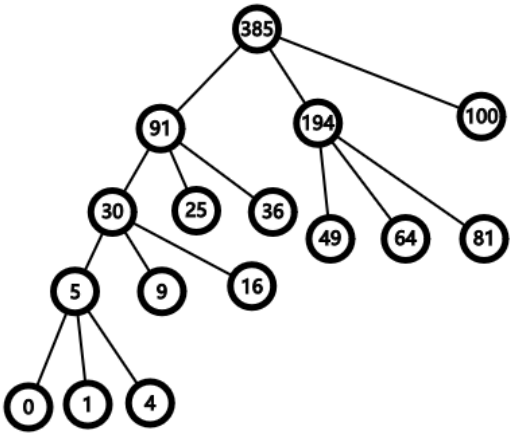
$$f(2^{w+1}-1)+(w+1)+1=2(f(2^w-1)+w+1)=2^w(0+1+1)=2^{w+1}$$
$$\Rightarrow f(2^w-1)=2^w-w-1.$$

因此 $f(n)=n+\mathcal{O}(\log n)$, $C_0=1$.

5.

类似 Huffman 树的构造, 首先在权值集合中补充若干个 0 使得 $m\equiv 1 \pmod{k-1}$, 这样总能使非叶结点有 k 个孩子. 维护一个小根堆, 每次从堆中连续取 k 个元素, 新增树结点连向这 k 个元素所代表的结点, 然后把新增结点压入堆, 权值为 k 个元素的权值之和.

对于题中权值集合, 补充一个 0, 对 $[0,1,4,9,16,25,36,49,64,81,100]$ 构造如下:



加权路径长度和为 705.